



第2章 运算方法和运算器II





2.3 定点乘法运算





乘除法运算的机器实现方法

1. 完全软件实现

不设乘除法运算的硬件电路，而是由软件利用运算器中的加法和移位寄存器实现乘、除法运算

2. 加法器+硬件辅助电路实现

利用运算器中的加法器硬件电路和移位寄存器，再设计必要的扩展电路，可以用硬件通过加法和移位操作实现乘、除法运算

3. 专用乘、除法器实现

在运算器中除了设置加法器之外，采用专用硬件电路实现高速乘、除法部件，直接完成乘、除法运算





定点乘法运算算法

- 原码一位乘法运算
- 无符号的阵列乘法
- 有符号的阵列乘法





原码一位乘法运算

令： $[X]_{\text{原}} = X_f . X_1 X_2 X_3 \dots X_n$

$[Y]_{\text{原}} = Y_f . Y_1 Y_2 Y_3 \dots Y_n$

则

$$\begin{aligned} [X * Y]_{\text{原}} &= (X_f \oplus Y_f) + (|X| * |Y|) \\ &= (X_f \oplus Y_f) + (0.X_1 X_2 \dots X_n \cdot 0.Y_1 Y_2 \dots Y_n) \end{aligned}$$

积的符号：被乘数与乘数两符号的异或值

积的数值：被乘数与乘数两数的绝对值之积





二进制乘法

例. 0.1101×1.1011

$$\begin{array}{r}
 0.1101 \\
 \times 0.1011 \\
 \hline
 1101 \\
 1101 \\
 0000 \\
 + 1101 \\
 \hline
 0.10001111
 \end{array}$$

加符号 **1.10001111**

基本算法描述:

- ◆ 若乘数的当前位为1, 则将被乘数和部分积求和
 - ◆ 若乘数的当前位为0, 则跳过
 - ◆ 将部分积移位
 - ◆ 所有位都完成后, 部分积即为最终结果
- N 位乘数 $\times$ M 位被乘数
 $\Rightarrow N+M$ 位的积
- 乘法显然比加法更复杂...
 - ◆ 但比十进制乘法要简单





部分积

令： $y = 0.y_1y_2\ldots y_n$

$$\begin{aligned}
 \text{则： } xy &= x \times (y_1 \times 2^{-1} + y_2 \times 2^{-2} + \ldots + y_n \times 2^{-n}) \\
 &= 2^{-1} \times (xy_1 + xy_2 \times 2^{-1} + \ldots + xy_n \times 2^{-n+1}) \\
 &= 2^{-1} \times (xy_1 + 2^{-1} (xy_2 + 2^{-1} (xy_3 + \ldots + \\
 &\qquad\qquad\qquad 2^{-1}(xy_n + 0)))
 \end{aligned}$$

令： $z_0 = 0$

$$z_1 = 2^{-1} (xy_n + z_0)$$

$$z_2 = 2^{-1} (xy_{n-1} + z_1)$$

.....

$$z_i = 2^{-1} (xy_{n-i+1} + z_{i-1})$$

$$z_n = 2^{-1} (xy_1 + z_{n-1}) = xy$$

部分积





A 部分积Z 乘数Y **C**

	00 0000	1 0 1 <u>1</u>	
+X	00 1101		
	00 1101		
右移一位→	00 0110	1 1 0 <u>1</u>	1(丢弃)
+X	00 1101		
	01 0011		
右移一位→	00 1001	1 1 1 <u>0</u>	1(丢弃)
+0	00 0000		
	00 1001		
右移一位→	00 0100	1 1 1 <u>1</u>	0(丢弃)
+X	00 1101		
	01 0001		
右移一位→	00 1000	1 1 1 1	1(丢弃)
	乘积高位	乘积低位	

B 00.1101 被乘数X

$$z_0 = 0, y_4 = 1$$

$$xy_4 + z_0$$

$$z_1 = 2^{-1}(xy_4 + z_0)$$

例: 设 X = 0.1101

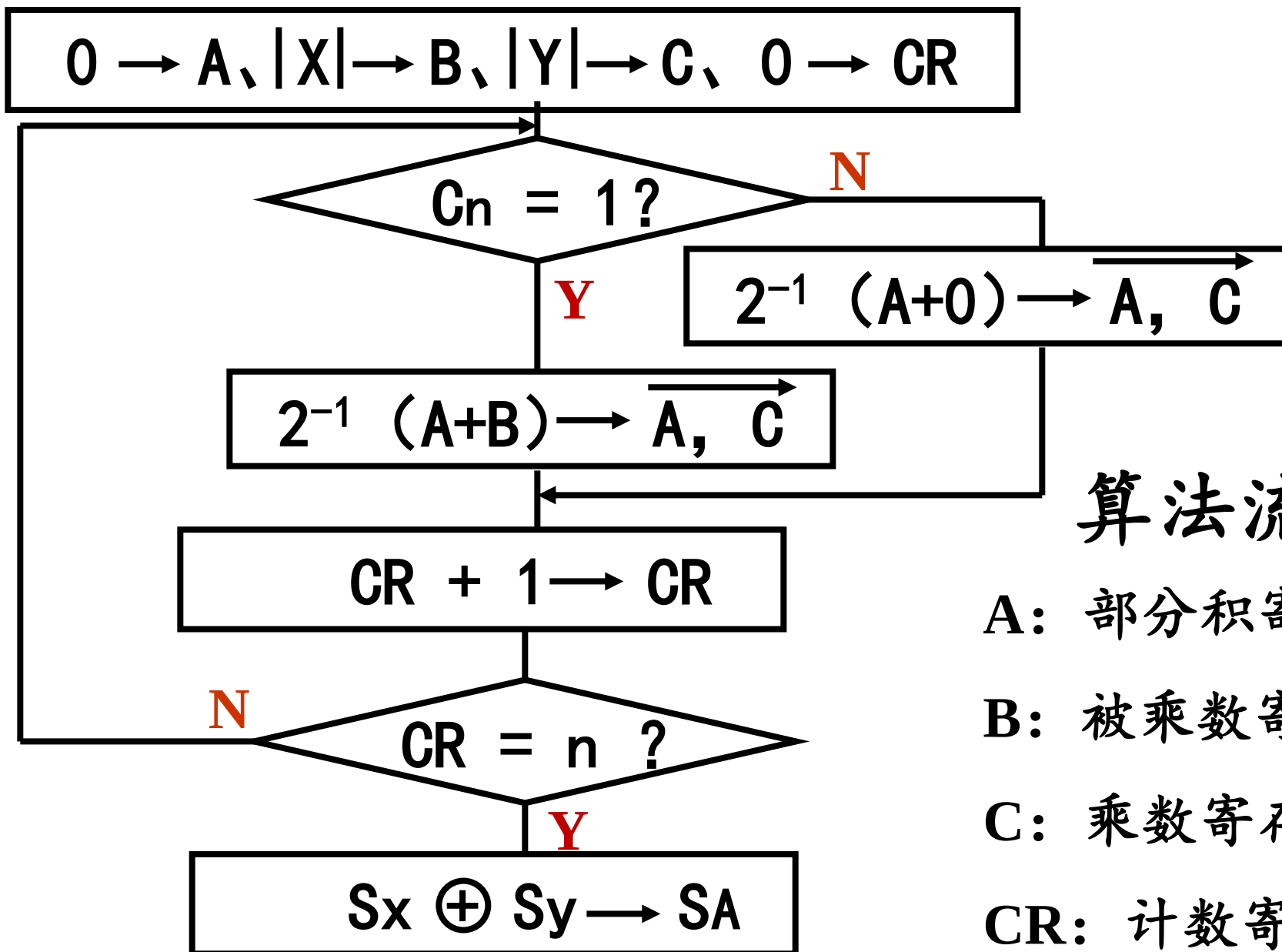
Y = 0.1011

求 X × Y

$$X \times Y = 0.10001111$$

移位4次, 运算完成





算法流程

A: 部分积寄存器

B: 被乘数寄存器

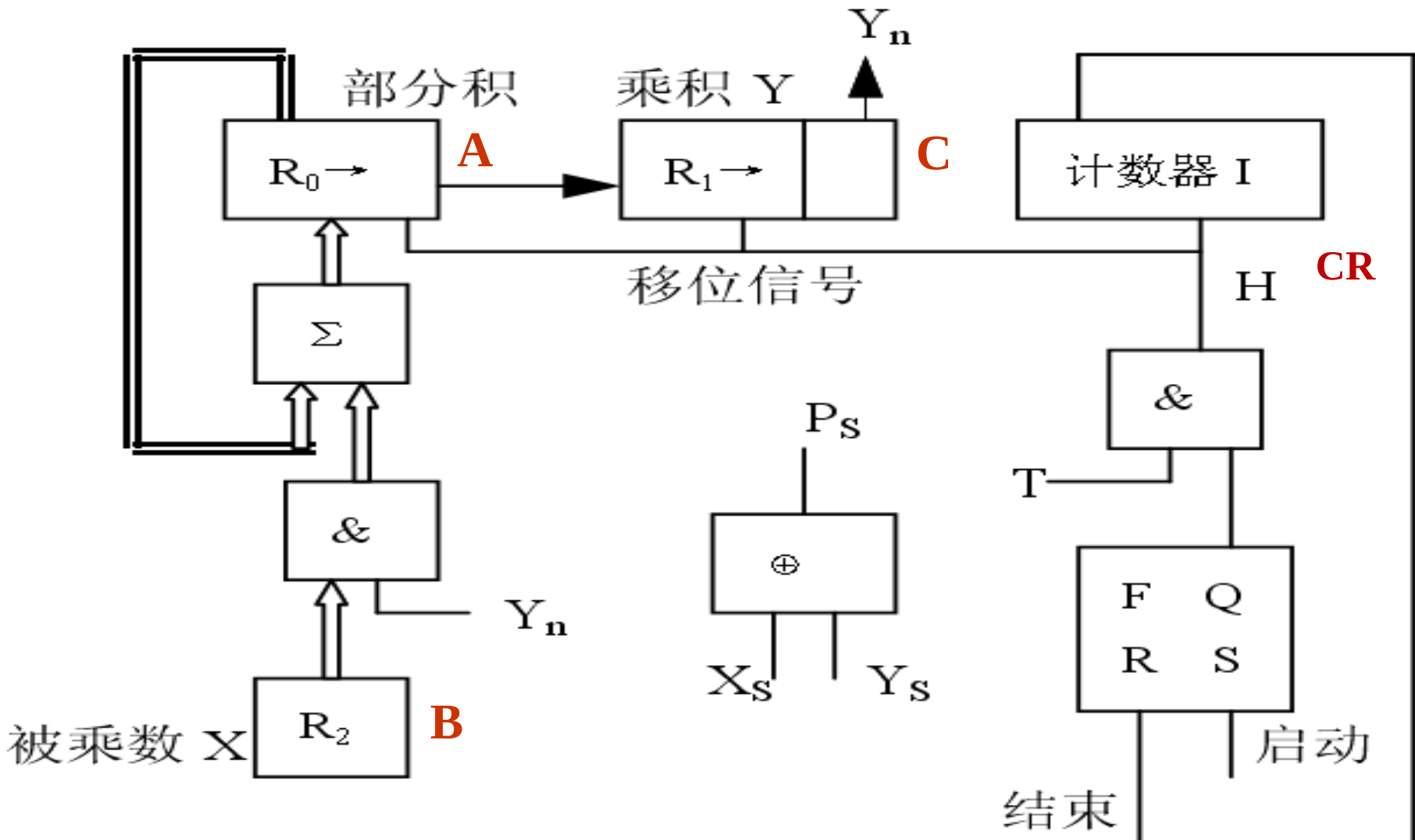
C: 乘数寄存器

CR: 计数寄存器

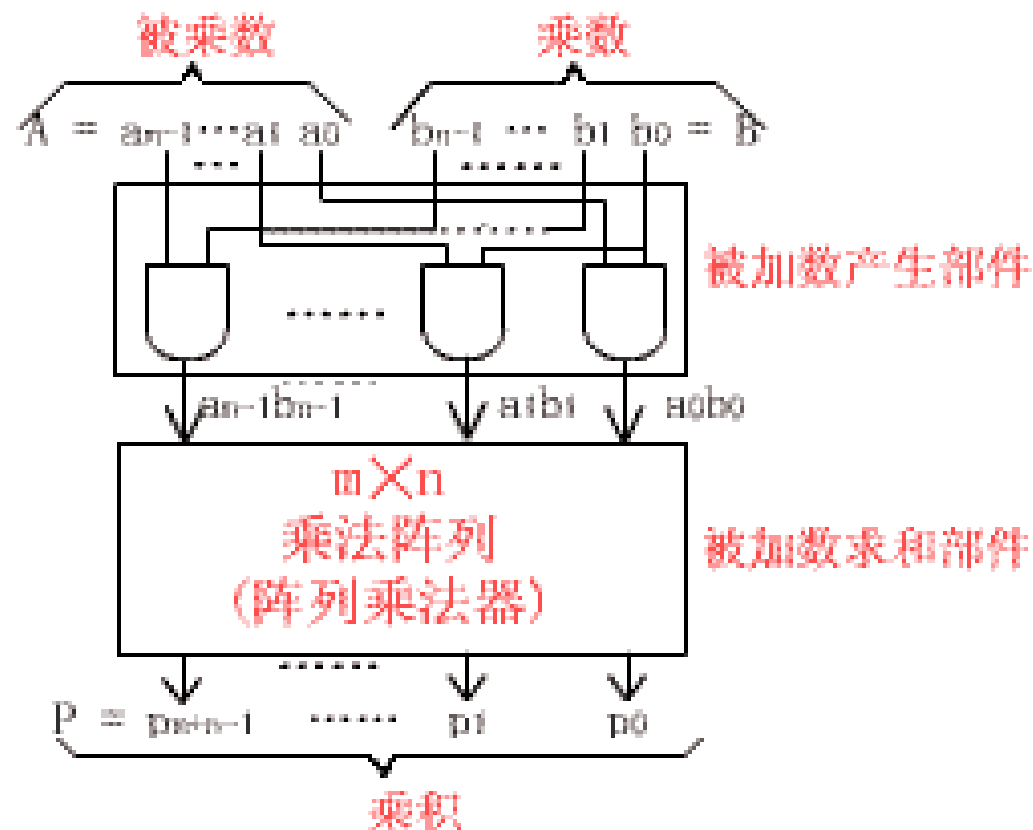




原码一位乘法的硬件电路

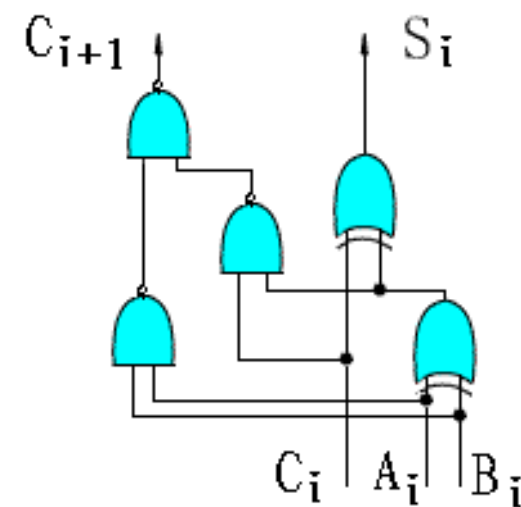
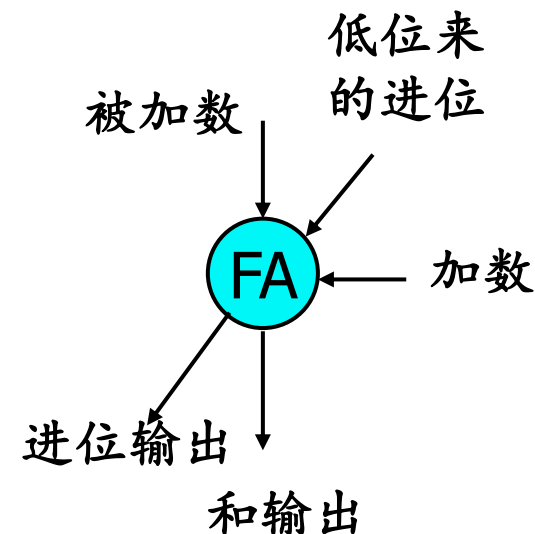
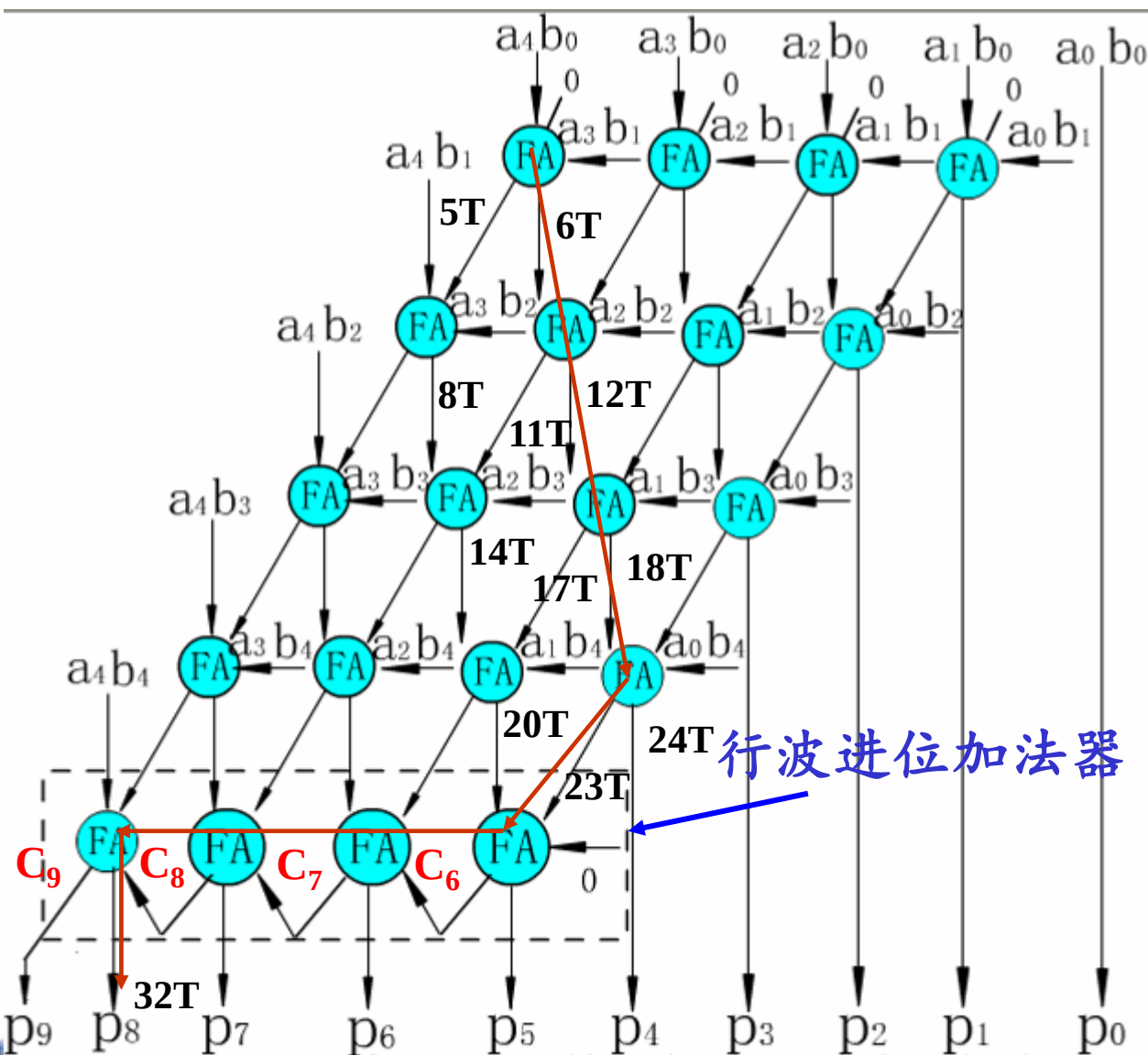


$m \times n$ 位不带符号的阵列乘法器逻辑图





5位×5位无符号阵列乘法器的求和部件





时间计算

- 实现 n 位 $\times n$ 位不带符号的原码阵列乘法器，需要 $n(n-1)$ 个全加器和 n^2 个“与”门
- n 位 $\times n$ 位不带符号的原码阵列乘法器总的乘法时间估算为：

$$\begin{aligned}t_m &= T_a + (n-1) \times 6T + (n-1) \times T_f \\&= T + (n-1) \times 6T + (n-1) \times T_f \\&= (8n-7)T\end{aligned}$$

- ◆ T_a 是计算 $a_i b_i$ 的时间，为 $1T$
- ◆ T_f 是全加器的进位延迟，为 $2T$





例20

$$a_4b_0=1 \quad a_3b_0=1 \quad a_2b_0=0 \quad a_1b_0=1 \quad a_0b_0=1$$

$$a_4b_1=0 \quad a_3b_1=0 \quad a_2b_1=0 \quad a_1b_1=0 \quad a_0b_1=0$$

$$a_4b_2=1 \quad a_3b_2=1 \quad a_2b_2=0 \quad a_1b_2=1 \quad a_0b_2=0$$

$$a_4b_3=0 \quad a_3b_3=0 \quad a_2b_3=0 \quad a_1b_3=0 \quad a_0b_3=0$$

$$a_4b_4=1 \quad a_3b_4=1 \quad a_2b_4=0 \quad a_1b_4=1 \quad a_0b_4=1$$

$$\begin{array}{r}
 \times \qquad \qquad \qquad 1 \ 1 \ 0 \ 1 \ 1 = A(27_{10}) \\
 \qquad \qquad \qquad 1 \ 0 \ 1 \ 0 \ 1 = B(21_{10}) \\
 \hline
 \qquad \qquad \qquad 1 \ 1 \ 0 \ 1 \ 1 \\
 \qquad \qquad \qquad 0 \ 0 \ 0 \ 0 \ 0 \\
 \qquad \qquad \qquad 1 \ 1 \ 0 \ 1 \ 1 \\
 \qquad \qquad \qquad 0 \ 0 \ 0 \ 0 \ 0 \\
 + \qquad \qquad 1 \ 1 \ 0 \ 1 \ 1 \\
 \hline
 1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 = P
 \end{array}$$

$$P = p_9p_8p_7p_6p_5p_4p_3p_2p_1p_0 = 1000110111 (567_{10})$$





有符号数的阵列乘法

- 由于有符号数在机器中用补码表示，一种有符号数的阵列乘法器的运算方法如下：
 - ◆ 正数：尾数参加无符号数乘法器运算
 - ◆ 负数：将补码求其绝对值后参加无符号数乘法器运算
 - ◆ 符号位通过异或门得到乘积的符号
 - ◆ 若乘积为负数，需将乘积的绝对值经过求补电路得到积的补码

实质就是将补码转换为原码，用绝对值参与无符号数乘法器运算，再将乘积求补得到运算结果的补码表示，符号位单独处理





求补器电路图

■例：求补

1 010101 $\xrightarrow{\text{求补}}$ 1 101011; 1 101000 $\xrightarrow{\text{求补}}$ 1 011000

- ◆从数的最右端开始，从右向左直到找出第一个“1”，该“1”保持不变，以左的每一个位都求反

E是符号位

E=0时输出与输入相等

E=1时对其求补

$$C_{-1}=0$$

$$C_i = a_i + C_{i-1}$$

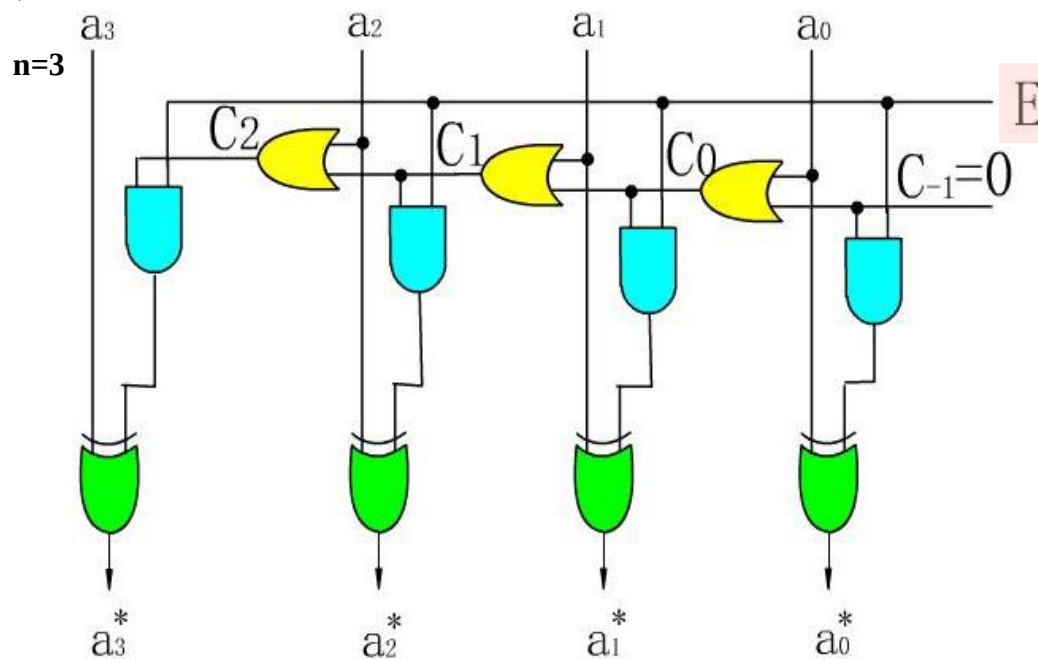
$$a_i^* = a_i \oplus EC_{i-1}$$

n+1位(不含符号位)的时间:

$$t_{TC} = n \cdot 2T + 2T + 3T$$

$$= (2n+5)T$$

假设：与门、或门延迟为2T，异或门延迟为3T



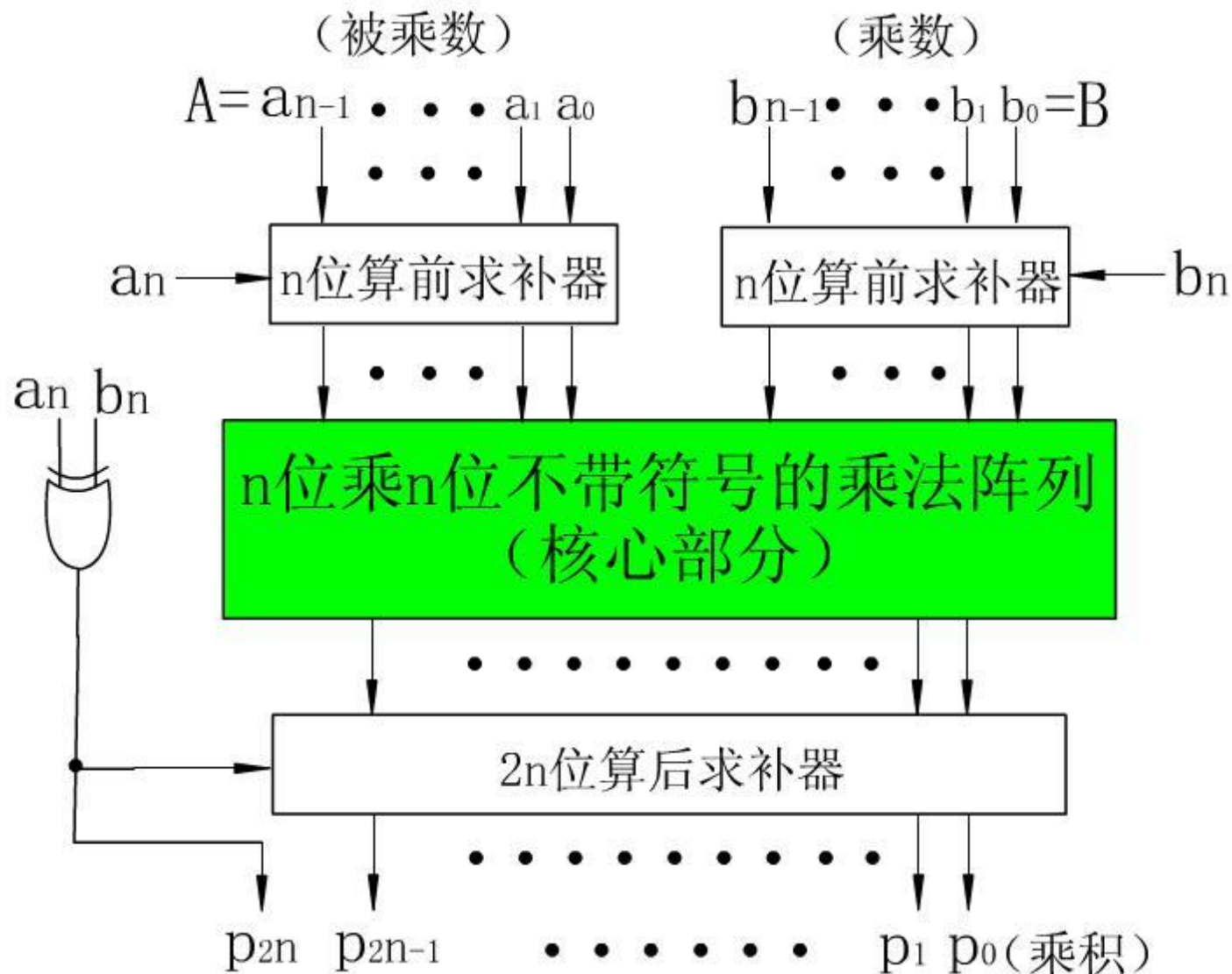
带求补器的阵列乘法器 (n+1位)

三个求补器:

■ 两个n位
算前求补器

■ 一个2n位
算后求补器

n×n位不
带符号的乘
法阵列





例21

设 $x = +15$, $y = -13$, 若机器用补码表示有符号数, 用带求补器的阵列乘法器求乘积 $x \times y$; 并用十进制数乘法进行验证。

$$[x]_{\text{补}} = 01111 \quad [y]_{\text{补}} = 10011$$

算前求补器输出 $|x| = 1111$, $|y| = 1101$

$$\begin{array}{r}
 \times \qquad \qquad \qquad 1 \ 1 \ 1 \ 1 \\
 \hline
 \qquad \qquad \qquad 1 \ 1 \ 0 \ 1 \\
 \qquad \qquad \qquad 1 \ 1 \ 1 \ 1 \\
 \qquad \qquad 0 \ 0 \ 0 \ 0 \\
 \qquad 1 \ 1 \ 1 \ 1 \\
 + \quad 1 \ 1 \ 1 \ 1 \\
 \hline
 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1
 \end{array}$$

符号位运算: $0 \oplus 1 = 1$

算后求补级输出并加上乘积符号位1, 得

$$[x \times y]_{\text{补}} = 100111101$$

二进制数真值: $x \times y = (-11000011)_2 = (-195)_{10}$

十进制数验证: $x \times y = 15 \times (-13) = -195$ 相等。





2.4 定点除法运算





原码除法运算原理

- 设有n位定点小数（定点整数也适用）

被除数x，其原码为 $[x]_{\text{原}} = x_f \cdot x_{n-1} \dots x_1 x_0$

除数y，其原码为 $[y]_{\text{原}} = y_f \cdot y_{n-1} \dots y_1 y_0$

则其商 $q = x \div y$ ，其原码为

$$[q]_{\text{原}} = (x_f \oplus y_f) + (0.x_{n-1} \dots x_1 x_0 \div 0.y_{n-1} \dots y_1 y_0)$$





手工除法运算过程

$x \div y?$, $x=0.1001$, $y=0.1011$

	0.1 1 0 1	商 q
0.1 0 1 1	0.1 0 0 1 0	$x(r_0)$ 被除数小于除数, 商0
	-0.01011	$2^{-1}y$ 除数右移1位, 减除数, 商1
	0.0 0 1 1 1 0	r_1 得余数 r_1
	-0.001011	$2^{-2}y$ 除数右移1位, 减除数, 商1
	0.0 0 0 0 1 1 0	r_2 得余数 r_2
	-0.0001011	$2^{-3}y$ 除数右移1位, 不减除数, 商0
	0.0 0 0 0 1 1 0 0	r_3 得余数 r_3
	-0.00001011	$2^{-4}y$ 除数右移1位, 减除数, 商1
	-0.00000001	r_4 得余数 r_4

- 商0还是商1? 人需比较后才能确定
- 余数末位补0后, 减去除数右移后的值, 导致加法器尾数逐渐增多, 最后要求加法器的位数必须为被除数的2倍





加减交替法（不恢复余数法）

- 判断是否够减：被除数或部分余数左移减去除数得新余数

$$r'_i = 2r_{i-1} - y$$
 - ◆ 若 $r'_i > 0$ ，则 i 位上商 1，将在下一步把 r'_i 左移减除数。因为：
 - $r_i = r'_i$
 - $r_{i+1} = 2r_i - y = 2r'_i - y$
 - ◆ 若 $r'_i < 0$ ，则 i 位上商 0，将在下一步把 r'_i 左移加除数。因为：
 - $r_i = r'_i + y$
 - $r_{i+1} = 2r_i - y = 2(r'_i + y) - y = 2r'_i + y$

即：将恢复余数与下一步余数左移减除数合并为加余数
 $2r'_i + y$ 也可理解为：商 0 后， $2r'_i$ 中实际多减了 $2y$ ，那么计算 r_{i+1} 时加 y 就是减除数的结果
- 如果最终的余数是负数，需“纠余”得到除法运算的余数：

$$r_n = r'_n + y$$





例:

$X=0.01011$

$Y=0.01101$

被除数(余数)

商

```

      0 0 1 0 1 1
+ )  1 1 0 0 1 1
-----
  0 1 1 1 1 0
    1 1 1 1 0 0
    +) 0 0 1 1 0 1
    -----
      1 0 0 1 0 0 1
        0 1 0 0 1 0
        +) 1 1 0 0 1 1
        -----
          1 0 0 0 1 0 1
            0 0 1 0 1 0
            +) 1 1 0 0 1 1
            -----
              0 1 1 1 0 1 0
                1 1 1 0 1 0
                +) 0 0 1 1 0 1
                -----
                  1 0 0 0 1 1 1
  
```

补码

0 0 0 0 0

0 0 0 0 0

0 0 0 0 0

0 0 0 0 1

0 0 0 1 0

0 0 0 1 1

0 0 1 1 0

0 0 1 1 0

0 1 1 0 0

0 1 1 0 1

开始情形

-Y

<0, 商0

左移1位

+Y

>0, 商1

左移1位

-Y

>0, 商1

左移1位

-Y

<0, 商0

左移1位

+Y

>0, 商1





可控加法/减法 (CAS) 单元

- 当输入线 $P=0$ 时, CAS 做加法运算
- 当输入线 $P=1$ 时, CAS 做减法运算

$$S_i = A_i \oplus (B_i \oplus P) \oplus C_i$$

$$C_{i+1} = (A_i + C_i)(B_i \oplus P) + A_i C_i$$

当 $P=0$ 时:

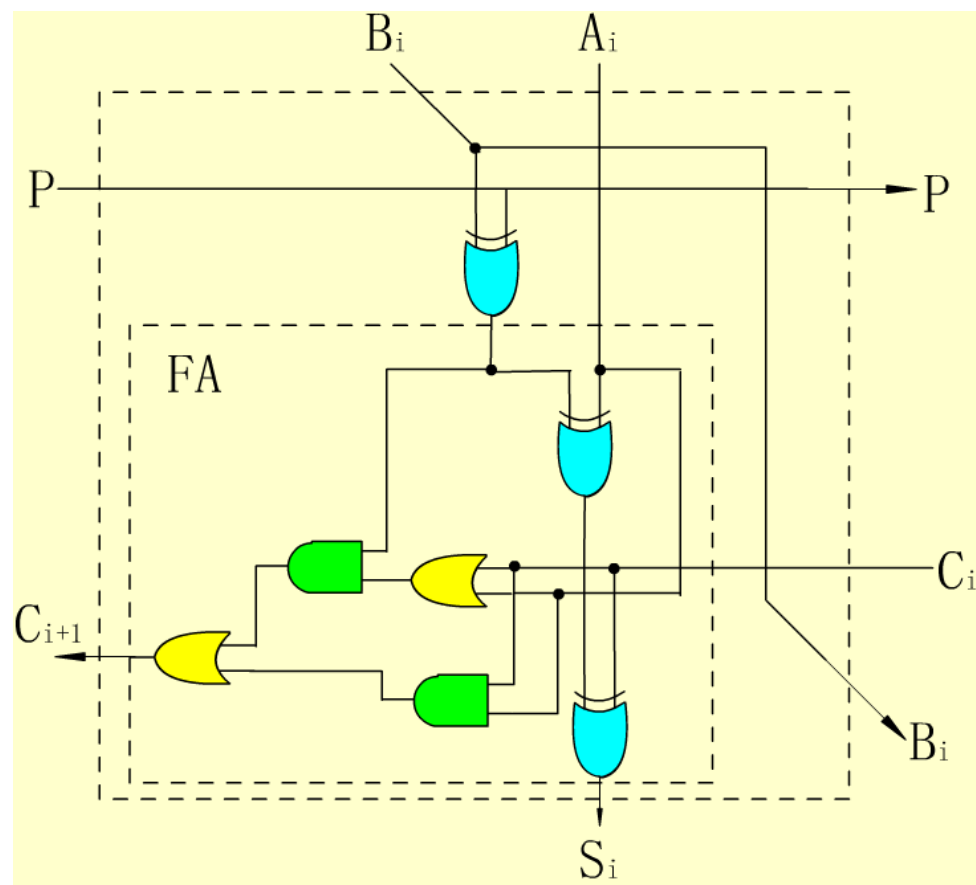
$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + B_i C_i + A_i C_i$$

当 $P=1$ 时:

$$S_i = A_i \oplus \bar{B}_i \oplus C_i$$

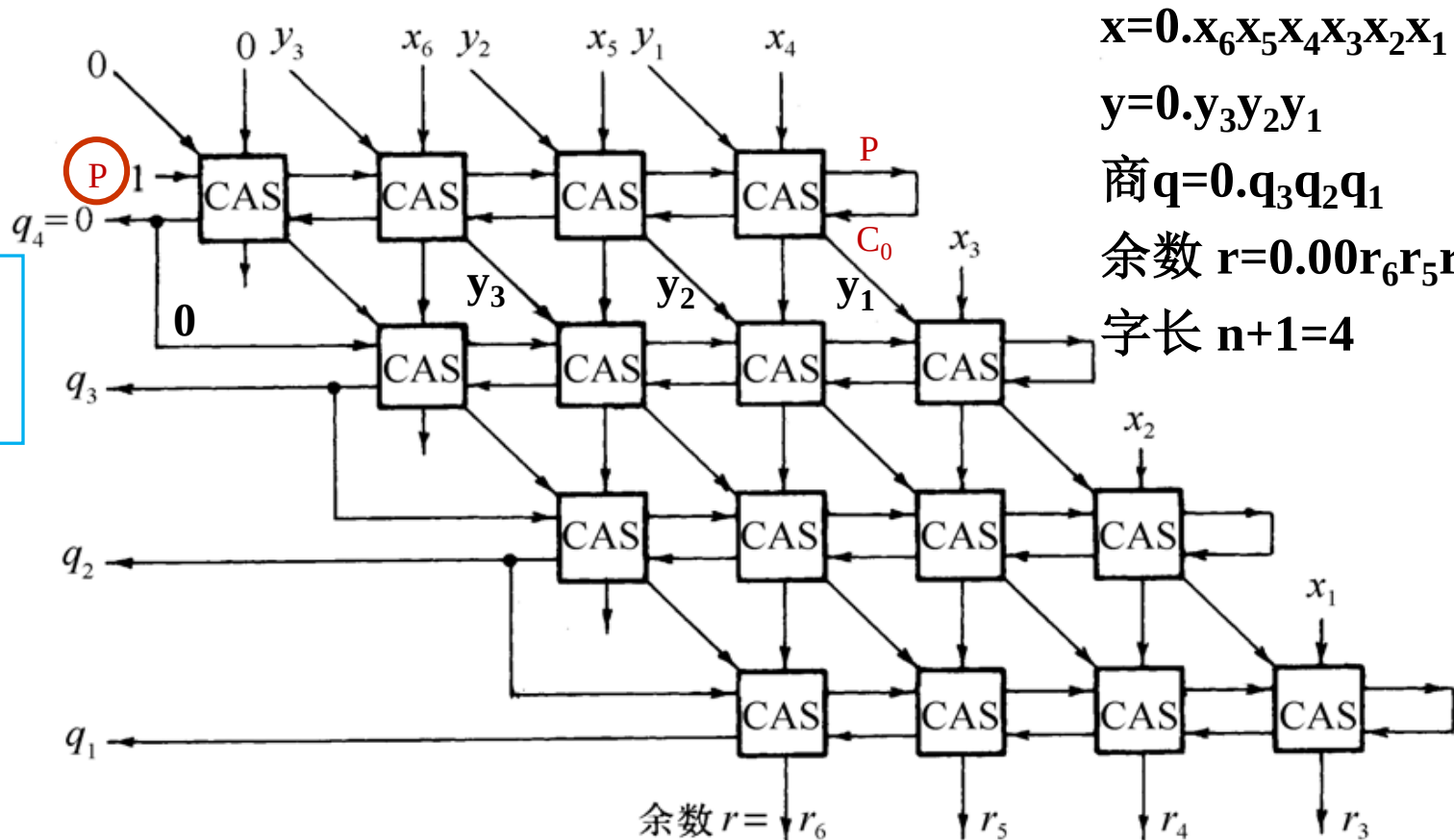
$$C_{i+1} = A_i \bar{B}_i + \bar{B}_i C_i + A_i C_i$$



$C_{i+1}=0$ 表示有借位



加减交替阵列除法器



进位输出表示部分余数的符号
0-负数、1-正数

加减交替法

前提：除数大于被除数，意味着商不会 ≥ 1



时间延迟

对一个 $2n$ 位（小数部分）除以 n 位的不恢复余数阵列除法器

- $(n+1)^2$ 个CAS单元

- CAS单元的进位延迟为 $3T$

除法的执行时间 $t = (n+1)^2 3T$
 $= 3(n+1)^2 T$





2.6 浮点数的机器表示与运算





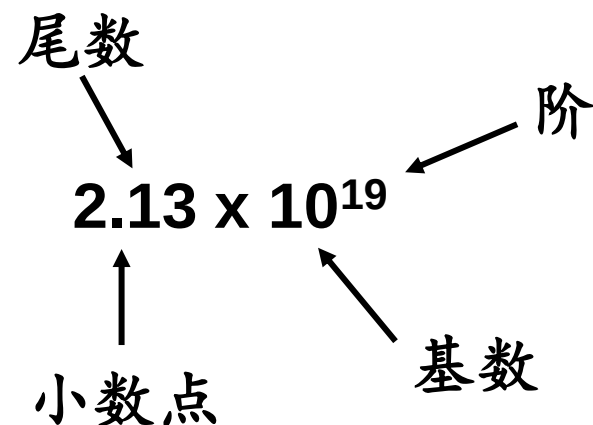
浮点数的表示方法（1）

- 任意一个十进制数N可以表示成

$$N = M \times 10^E$$

- 一个任意进制数N可以表示成

$$N = M \times R^E$$



其中，M称为浮点数的**尾数**，是一个小数；E称为浮点数的**指数或阶**，是一个整数；R称为**基数**，在二进制机器中通常规定R为2，8或16

- 一个数的小数点位置随指数取值不同而在一定范围内浮动，这种表示法称为浮点表示法

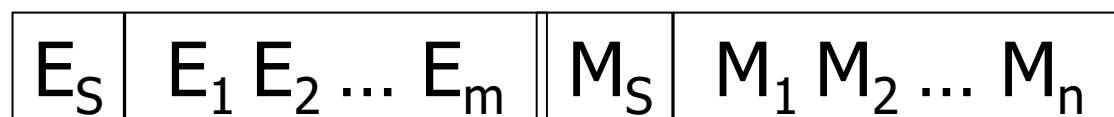




浮点数的表示方法 (2)

■ 机器浮点数的组成

- ◆ 尾数通常为纯小数，常用**原码**或**补码**表示。尾数的有效数字的位数决定了浮点数的表示精度
- ◆ 指数为整数，也称为阶码，常用**移码**或**补码**表示，阶码的位数决定了浮点数的表示范围



阶符

阶码

数符

尾数

若尾数和阶码均采用**原码**，**非规格化**表示方式，则：

最大正数 $(1 - 2^{-n}) \times 2^{2^m-1}$

最大负数 $-2^{-n} \times 2^{-(2^m-1)}$

最小正数 $2^{-n} \times 2^{-(2^m-1)}$

最小负数 $-(1 - 2^{-n}) \times 2^{2^m-1}$





浮点数的表示方法 (3)

■ 浮点数的规格化形式

- ◆ 由于 1.01×2^3 可写成 $0.10\underline{1} \times 2^4$ 、 $0.01\underline{01} \times 2^5 \dots$
- ◆ 为了充分利用尾数的有效位数、提高运算精度，规定浮点数规格化形式是必要的
- ◆ 对于基数为 R 的规格化浮点数，定义其尾数的绝对值范围

$$1/R \leq |M| < 1$$

- ◆ 如果 $R=2$ ，则

$$1/2 \leq |M| < 1$$

对任一浮点数，其规格化形式是唯一的

若尾数用原码表示时，则尾数的最高位总为1

故，最小正数 $2^{-n} \times 2^{-(2^m-1)}$ 规格化形式为 $2^{-1} \times 2^{-(2^m-1)}$

最大负数 $-2^{-n} \times 2^{-(2^m-1)}$ 规格化形式为 $-2^{-1} \times 2^{-(2^m-1)}$





原码、补码和反码的规格化数

正数	规格化形式	负数	规格化形式
真值	0.1xxx...xxx	真值	-0.1xxx...xxx
原码	0.1xxx...xxx	原码	1.1xxx...xxx
补码	0.1xxx...xxx	补码	1.0xxx...xxx
反码	0.1xxx...xxx	反码	1.0xxx...xxx

特例:

$$x = -1/2 = -0.100 \dots 0 \Rightarrow [x]_{\text{原}} = 1.100 \dots 0$$

$[x]_{\text{补}} = \mathbf{1.1}00 \dots 0$, 但在补码中不是规格化数

而当 $x = -1$ 时 $\Rightarrow [x]_{\text{补}} = \mathbf{1.0}00 \dots 0$, 是规格化数

即若 M 为负数, 其补码的规格化范围: $-1 \leq M < -1/2$





例题

某浮点数字长32位，其中阶码8位，以2为底，补码表示；尾数为纯小数，24位（含1位数符），补码表示。现有一浮点数为C18F1234，求它所表示的二进制真值是多少？

阶码 (8位)

数符

尾数 (23位)

解：展开C18F1234，得：

1100 0001 **1**000 1111 0001 0010 0011 0100

阶码原码：1011 1111，十进制：-63

尾数原码：1.111 0000 1110 1101 1100 1100

二进制真值： $-0.11100001110110111001100 \times 2^{-63}$

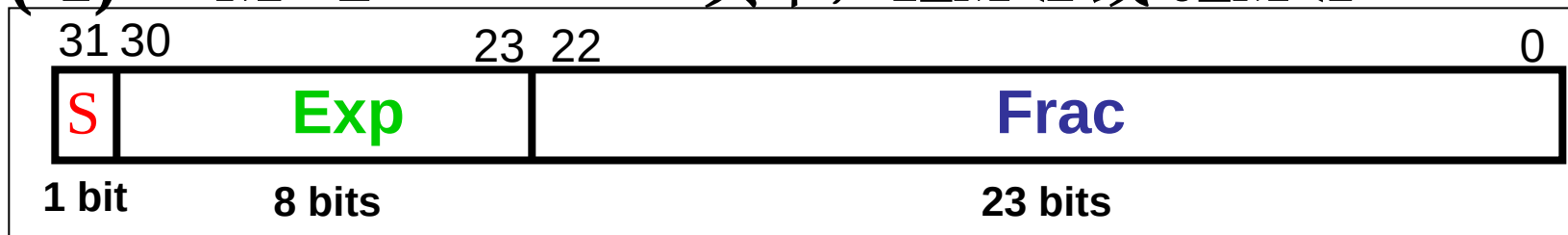




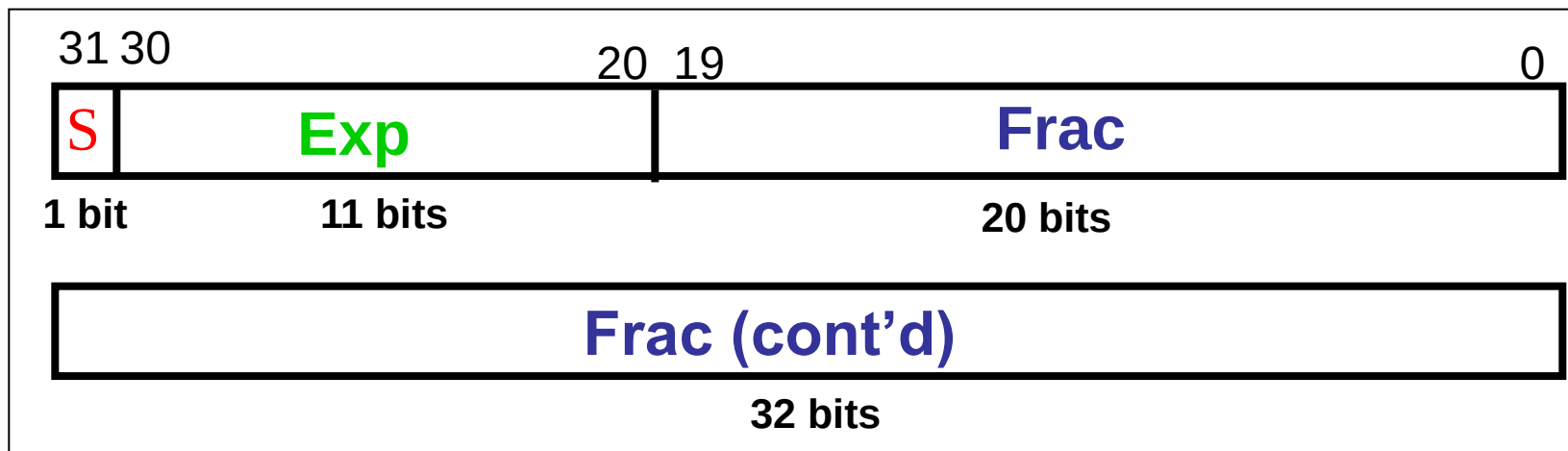
IEEE 754浮点数标准

$(-1)^S * M * 2^{\text{Exp}-\text{Bias}}$ 其中, $1 \leq M < 2$ 或 $0 \leq M < 1$

单精度
float x;



双精度
double y;



S: 浮点数的符号位, 0表示正数, 1表示负数

Frac: 小数字段, 用**原码**表示, 小数点放在尾数域的最前面。规格化数中: $M = 1 + \text{Frac}$, 即: **隐含最高位1**

Exp: 阶码, 用**移码**来表示

Bias: 偏移值 $\begin{cases} 127 & (\text{单精度}) \\ 1023 & (\text{双精度}) \end{cases}$





阶的移码表示

- IEEE 754中，浮点数的阶E采用**移码**表示
- 移码：**规格化数**，在真正的阶e上加一个规定的值
 - ◆ 对单精度浮点数： $\text{Exp} = e + \text{Bias} = e + 127$
 - ◆ 对双精度浮点数： $\text{Exp} = e + \text{Bias} = e + 1023$
- 最小阶： 00000001_2 1_{10}
- 最大阶： 11111110_2 254_{10} (单精度)
- 例如： $1.0 * 2^{-1}$ 在机器中的表示如下：

0 0111 1110	0000 0000 0000 0000 0000 0000
-------------	-------------------------------

真值计算： $(-1)^S * (1 + \text{Frac}) * 2^{(E - \text{Bias})}$





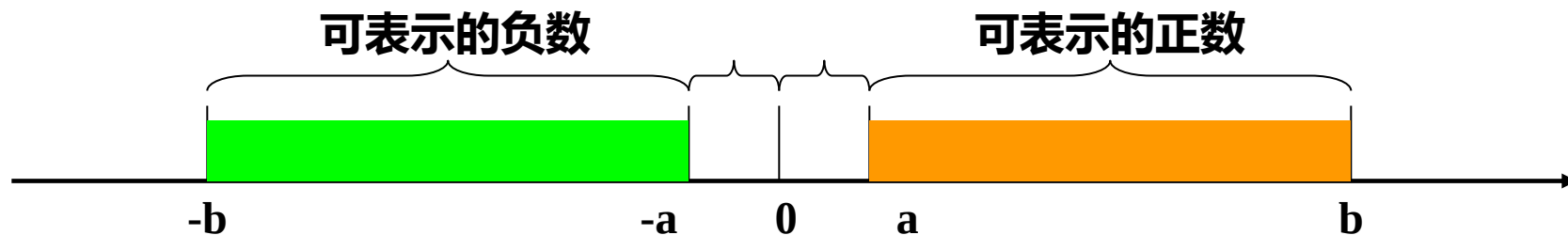
小数字段Frac

- 尾数用原码表示
- 规格化数表示，**隐含最高位1**
 - ◆ $M = 1 + \text{Frac}$, $1 \leq M < 2$
- 非规格化数表示
 - ◆ $M = \text{Frac}$, $0 \leq M < 1$
- 单精度为：1 + 23 位，双精度为 1 + 52 位
- 含义：
 - ◆ 十进制： $1.6732 = (1 \times 10^0) + (6 \times 10^{-1}) + (7 \times 10^{-2}) + (3 \times 10^{-3}) + (2 \times 10^{-4})$
 - ◆ 二进制： $1.1001 = (1 \times 2^0) + (1 \times 2^{-1}) + (0 \times 2^{-2}) + (0 \times 2^{-3}) + (1 \times 2^{-4})$





规格化数表示的数值范围（单精度）



- 最小正数a: $M=1.00\dots_2$, $\text{Exp}=1$
- 最大正数b: $M=1.1\dots 11_2$, $\text{Exp}=254$

◆ $a = 1.0\dots_2 * 2^{1-127} = 2^{-126}$

◆ $b = 1.1\dots 11_2 * 2^{254-127}$

$$= \underbrace{0.11\dots 11_2}_{24\text{个}1} * 2^{128}$$

$$= (1 - 2^{-24}) * 2^{128}$$

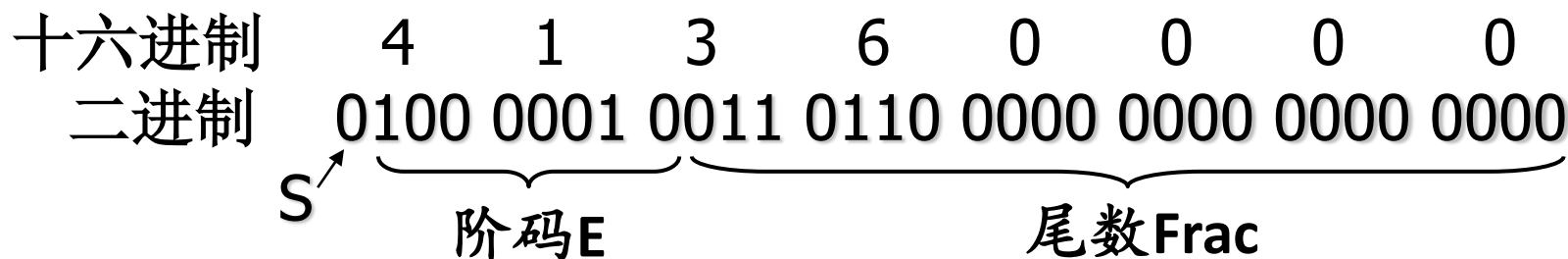




浮点数的二 $\Rightarrow$ 十进制转换

若32位浮点数x在计算机中存储的数为 $(41360000)_{16}$ ，求其十进制值

解：首先将十六进制数展开，得到二进制格式



$$x = (-1)^S \times 1.\text{Frac} \times 2^{\text{Exp} - 127}$$

$$x = (-1)^0 \times (1.011011) \times 2^{10000010 - 01111111}$$

$$x = +(1.011011) \times 2^3 = +1011.011$$

$$= 11 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}$$

$$= 11.375$$





浮点数十 $\Rightarrow$ 二进制转换 (1)

- 把纯小数化为分数后，如果分母是2的整数次方，则转换结果是准确的；否则转换结果是近似的。
- 如： - 0.75的二进制
 - ◆ $-0.75 = -3/4 \Rightarrow -11_2 \times 2^{-2} = -0.11_2$
 - ◆ 规格化为： $-1.1_2 \times 2^{-1}$
 - ◆ $(-1)^1 \times (1 + .100\ 0000 \dots 0000) \times 2^{-1}$

1	0111 1110	1000 0000 0000 0000 0000 0000
---	-----------	-------------------------------





浮点数十 $\Rightarrow$ 二进制转换 (2)

- 将小数化为分数，若分母**不是**2的整数倍，转换方法为：
 - ◆ 求出足够多的有效位
 - ◆ 根据精度要求截断多余的位。
 - ◆ 按标准要求给出符号位、阶和尾数。
- 如： 求-2.15的二进制

0.15	0.30	0.60	0.20	0.40	0.80	$\dots$
$\times 2$	$\times 2$	$\times 2$	$\times 2$	$\times 2$	$\times 2$	
0.30	0.60	1.20	0.40	0.80	1.60	

$\Rightarrow$ - 10.00100110011001100110011001

- 规格化 - 1.0001001100110011001100110011001... $\times 2^1$
- 阶 $1 + 127 = 128 = 10000000_2$

1	10000000	0001 0011 0011 0011 0011 001
---	----------	------------------------------





特殊的浮点数值

- IEEE浮点标准中，阶码0、阶码 $(2^8 - 1)$ 或 $(2^{11} - 1)$ 被保留，用作特殊用途

特殊值	阶	小数（尾数）
± 0	0000...0000	0
非规格化数	0000...0000	非0
NaN (Not a Number)	1111...1111	非0
$\pm \infty$	1111...1111	0

例如： $\text{sqrt}(-1)$, $\infty - \infty$, $\infty \times 0$ 等就是NaN





非规格化数表示范围（单精度）

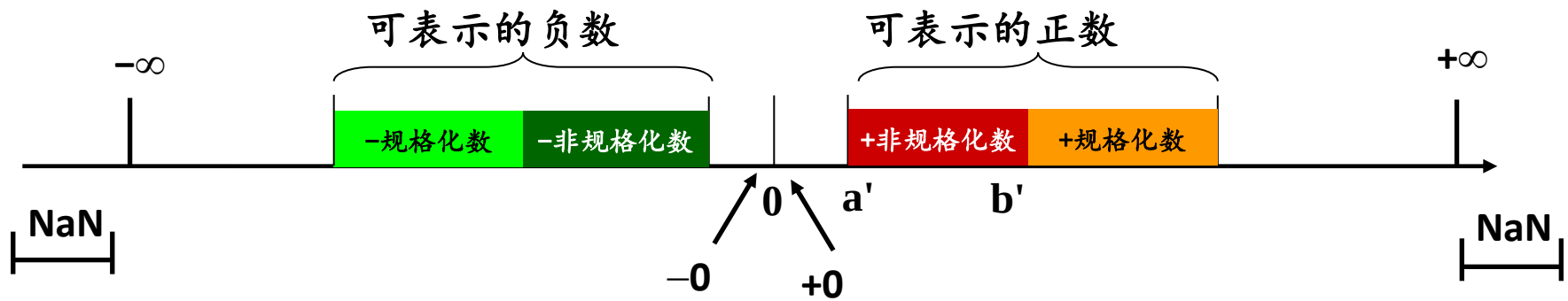
非规格化数 $\Rightarrow \text{Exp}=0$

- $M=\text{Frac}$ ，没有隐含的前导1
- $e = 1 - \text{Bias} = 1 - 127 = -126$ ，注：不是0 - 127，保证了非规格化值到规格化值的平滑过渡
- 最小的正数： $M=0.00\dots1_2$
 - ◆ $a' = 0.0\dots1_2 * 2^{1-127}$
$$= 2^{-23} * 2^{-126}$$
$$= 2^{-149}$$
- 最大的正数： $M=0.11\dots1_2$
 - ◆ $b' = 0.11\dots1_2 * 2^{1-127} = 0.11\dots1_2 * 2^{-126}$
$$= (1 - 2^{-23}) * 2^{-126}$$





浮点数表示的数值范围





IEEE 754 32位浮点数 小结

对32位浮点数N:

- 若 $0 < E < 255$, 则

$$N = (-1)^s \times 1.\text{Frac} \times 2^{E-127}, \text{规格化数表示}$$

- 若 $E = 0$ 且 $\text{Frac} = 0$, 则

$$N = (-1)^s 0, \text{机器+0、-0表示}$$

- 若 $E = 0$ 且 $\text{Frac} \neq 0$, 则

$$N = (-1)^s \times 0.\text{Frac} \times 2^{1-127}, \text{非规格化数表示}$$

- 若 $E = 255$ 且 $\text{Frac} = 0$, 则

$$N = (-1)^s \infty \text{ (正无穷大, 负无穷大)}$$

- 若 $E = 255$ 且 $\text{Frac} \neq 0$, 则

$$N = \text{NaN}, \text{非数NaN}$$





例题 (1)

- 假设由S，E和M三个域组成的一个32位二进制数，所表示的非零规格化浮点数 x ，真值表示为（注：不是IEEE754格式）：

$$x = (-1)^s \times (1.M) \times 2^{E-128}$$

问：它所表示的规格化的最大正数、最小正数、最大负数、最小负数是多少？

(1)最大正数

0 **1111 1111** 111 1111 1111 1111 1111 1111

$$x = [1 + (1 - 2^{-23})] \times 2^{127}$$





例题 (2)

(2)最小正数

0 0000 0000 000 0000 0000 0000 0000 0000

$$x = 1.0 \times 2^{-128}$$

(3)最小负数

1 1111 1111 111 1111 1111 1111 1111 1111

$$x = -[1 + (1 - 2^{-23})] \times 2^{127}$$

(4)最大负数

1 0000 0000 000 0000 0000 0000 0000 0000

$$x = -1.0 \times 2^{-128}$$





动手做

float型整数据常用**IEEE754**单精度浮点格式表示，假设两个**float**型变量**x**和**y**分别在32为寄存器**f1**和**f2**中，若 $(f1) = CC900000H$ ， $(f2) = B0C00000H$ ，则**x**和**y**之间的关系为：

- A $x < y$ 且符号相同
- B $x < y$ 且符号不同
- C $x > y$ 且符号相同
- D $x > y$ 且符号不同





动手做

1、长度相同但格式不同的2种浮点数，假设前者阶码长、尾数短，后者阶码短、尾数长，其他规则均相同，则它们可表示的数的范围和精度为

- A.两者可表示的数的范围和精度相同
- B.前者可表示的数的范围大但精度低
- C.后者可表示的数的范围大且精度高
- D.前者可表示的数的范围大且精度高





浮点数加、减法

设有两个浮点数 $x = 2^{E_x} \times M_x$

$$y = 2^{E_y} \times M_y$$

$$x \pm y = (M_x 2^{E_x - E_y} \pm M_y) 2^{E_y}, \quad E_x \leq E_y$$

■ 算法

1. 检查操作数是否为0
2. 对阶，求阶差： $\Delta E = E_x - E_y$ ，使阶码小的数的尾数**右移** $|\Delta E|$ 位，其阶码取大的阶码值
3. 对尾数进行加、减运算
4. 规格化（包括阶码上溢或下溢的处理）
5. 舍入（可能再次规格化），





对阶

- 原则：小阶向大阶看齐（为什么？）
- 方法：
 - ◆ 逐位移位比较：小阶的尾数每次右移一位，其阶码加1，直到两数的阶码相等为止
 - ◆ 求阶差：
 - 右移的位数等于阶差 $\Delta E = E_x - E_y$
 - 哪个数移位由阶差的正负决定。
 - 若 $\Delta E = 0$, 表示两数阶码相等;
 - 若 $\Delta E > 0$, 表示 $E_x > E_y$; 则y移位;
 - 若 $\Delta E < 0$, 表示 $E_x < E_y$; 则x移位。





结果规格化

■ 左规：向左规格化

- ◆ 若尾数为原码时，应使结果为 $x.1xxxxx$
- ◆ 若尾数为补码时，应使尾数的最高位与符号位相反。即：若符号位与最高位相等，则应逐次使尾数左移，低位补0，并使阶码减1，直到符合规定。
- ◆ 而对IEEE754浮点格式，应使尾数变为1.M形式

■ 右规：向右规格化

- ◆ 若尾数求和的结果为 $01.x...x$ 或 $10.x...x$ ，应将运算结果右移以实现规格化表示
- ◆ 规则：尾数右移1位，阶码加1





IEEE754的舍入处理

四种舍入（Rounding）方式：

- **就近舍入（向偶数舍入）**：若尾数超出规定的23位的多余位数共3位（形如1.xxyabc），若abc取值为001、010、011，则舍去；若取值为101、110、111，则向最低有效位进位。但当取值为中间值100时，需进一步判断最低有效位y，若为0，则舍去；若为1，则进位；为中间值时舍入后结果始终为偶数。
- **朝0舍入**：即朝数轴原点方向舍入，就是把多余的位数简单的截去。截尾使取值的绝对值比原值的绝对值小。
- **朝 $+\infty$ 舍入（向上舍入）**：对于正数，当多余位不全为0则向最低有效位进1；而对负数则是简单的截尾。
- **朝 $-\infty$ 舍入（向下舍入）**：处理方法正好与朝 $+\infty$ 舍入情况相反。对于正数，当多余位不全为0则简单截尾；而对负数，则向最低有效位进1。

“就近舍入”为默认规则





溢出判断和处理

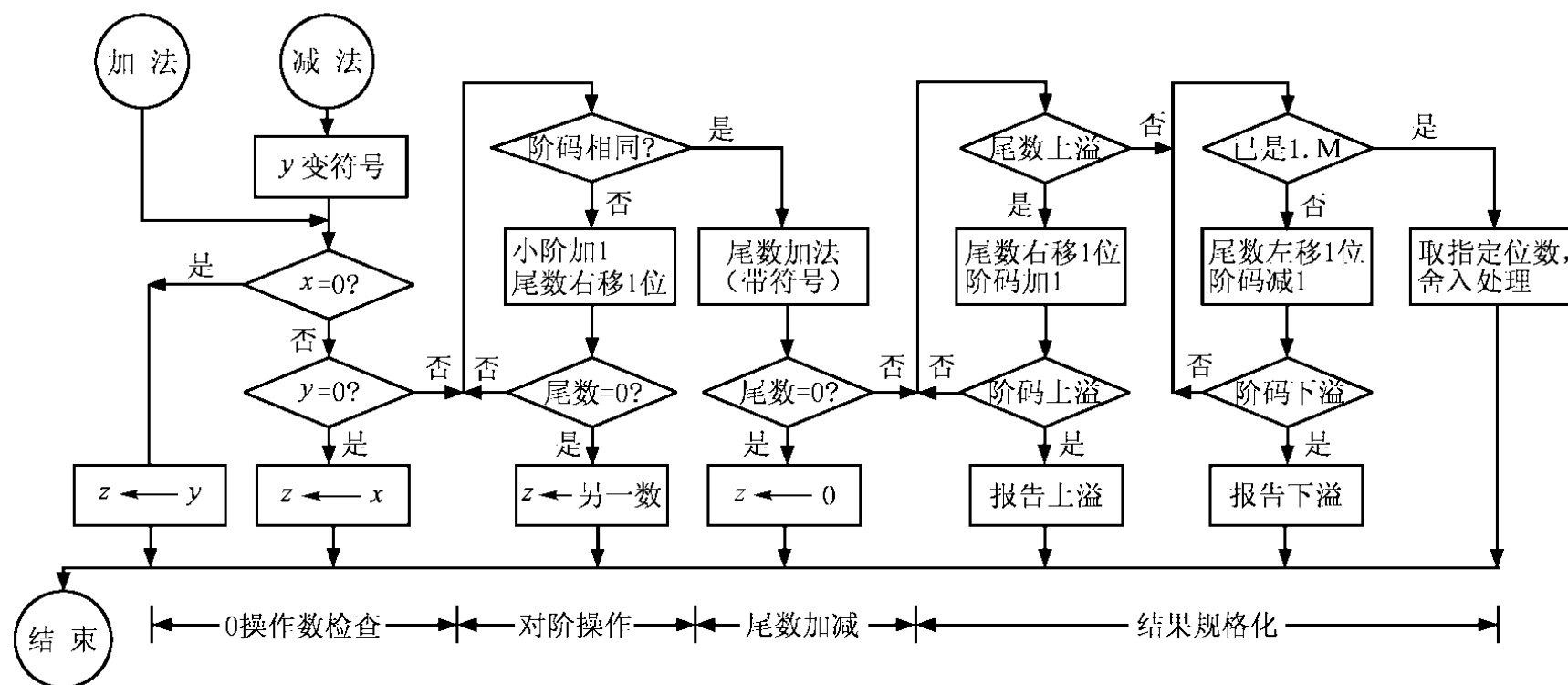
- 阶码上溢，超过阶码能表示的最大正指数值，一般将其认为是 $+\infty$ 和 $-\infty$
- 阶码下溢，超过阶码能表示的最小负指数值，一般将其认为是0
- 尾数上溢，两个同符号位的数相加，产生最高位的进位，需要将尾数右移，阶码加1
- 尾数下溢，尾数右移会使有效数字从最右端流出，最终导致尾数变为0





浮点加减运算操作流程

■ 分为四步完成

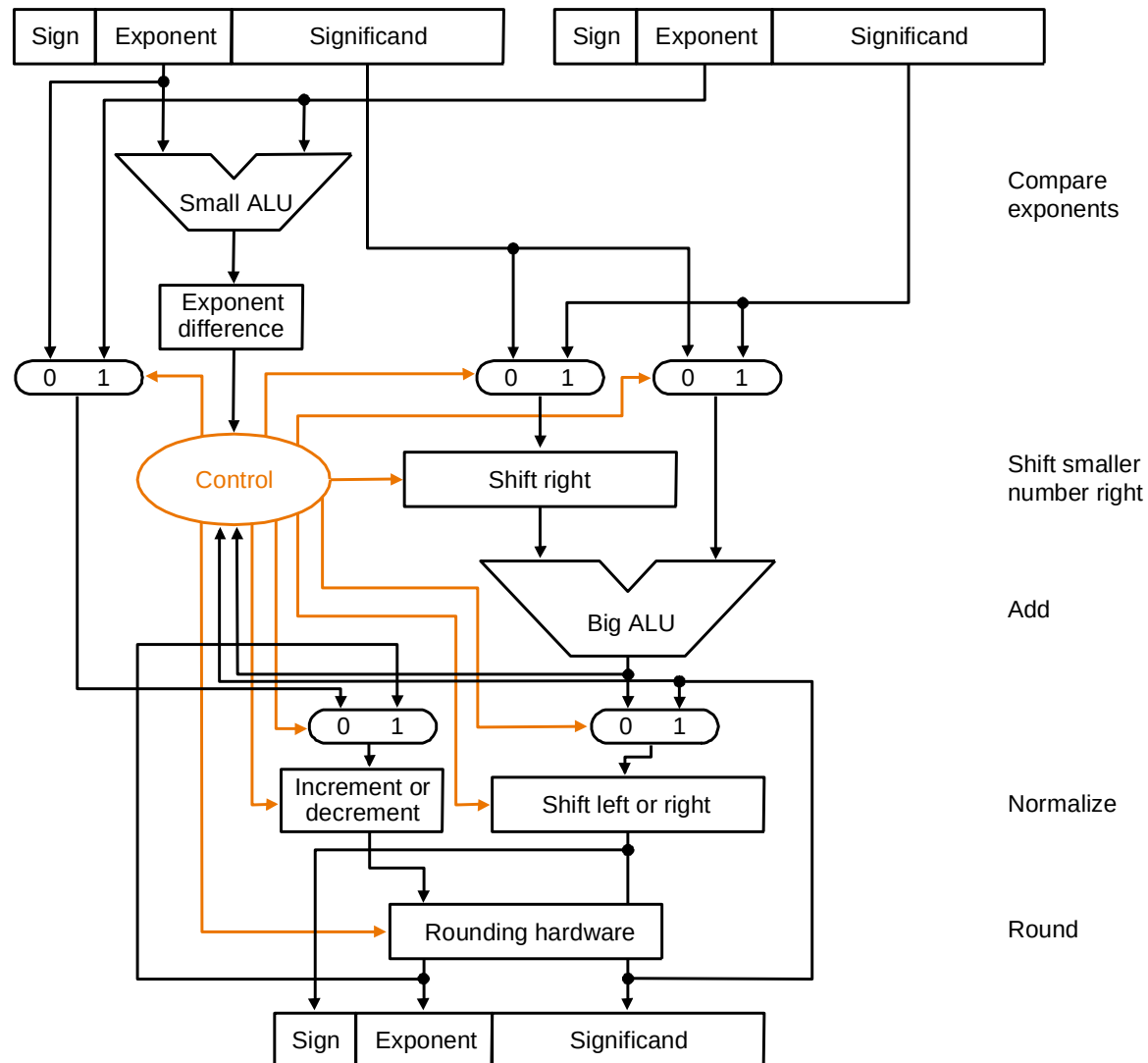


再观察： $(x+y) + z =? x + (y+z)$





浮点运算部件





例

■ 设 $x=0.5$, $y=-0.4375$, 假设尾数有效位为4位, 用二进制形式求 $(x+y)_{\text{浮}}$ 。

解 $x=0.5=0.1 \times 2^0 = 1.000 \times 2^{-1}$

$y=-0.4375 = -0.0111 \times 2^0 = -1.110 \times 2^{-2}$

第1步, 对阶 $y = -1.110 \times 2^{-2} = -0.111 \times 2^{-1}$

第2步, 尾数相加 $x+y = 1.000 + (-0.111) = 0.001$

第3步, 规格化 $x+y = 0.001 \times 2^{-1} = 1.000 \times 2^{-4}$

第4步, 检查上溢或下溢 指数-4用移码表示, 在合理范围

第5步, 舍入操作 $x+y = 1.000 \times 2^{-4}$, 尾数为4位, 无需舍入操作

最后结果: $x+y = 1.000 \times 2^{-4} = 0.0001000 = 0.0625$

十进制数验证: $x+y = 0.5 - 0.4375 = 0.0625$





浮点数乘、除法

设有两个浮点数 $x = 2^{E_x} \times M_x$

$$y = 2^{E_y} \times M_y$$

■ $x \times y = 2^{(E_x + E_y)} \times (M_x \times M_y)$

$$x \div y = 2^{(E_x - E_y)} \times (M_x \div M_y)$$

■ 算法

1. 检查操作数是否为0

2. 阶码加、减操作

乘: $E_x + E_y$, 除: $E_x - E_y$

3. 对尾数进行乘、除操作

4. 结果规格化 (包括阶码上溢或下溢的处理)

5. 舍入处理 (可能再次规格化)

6. 确定乘积符号





尾数处理

- 截断
 - 或舍入，即按某种规则进行修正
- 当尾数用原码表示时
- 方法1: 只要尾数最低为1或移出位中包含有1，使最低位置1
- 方法2: 0舍1入





举例

- 设基数 $R=10$, $x=10^{E_x} \times M_x=10^2 \times 0.4$,
 $y=10^{E_y} \times M_y=10^3 \times 0.2$, 用浮点法求 $x \times y=?$ $x \div y=?$

解: $E_x = 2, E_y = 3, M_x = +0.4, M_y = +0.2$

$$x \times y = 10^{(E_x+E_y)} \times (M_x \times M_y) = 10^{2+3} \times (0.4 \times 0.2) \\ = 8000$$

$$x \div y = 10^{(E_x-E_y)} \times (M_x \div M_y) = 10^{2-3} \times (0.4 \div 0.2) \\ = 0.2$$





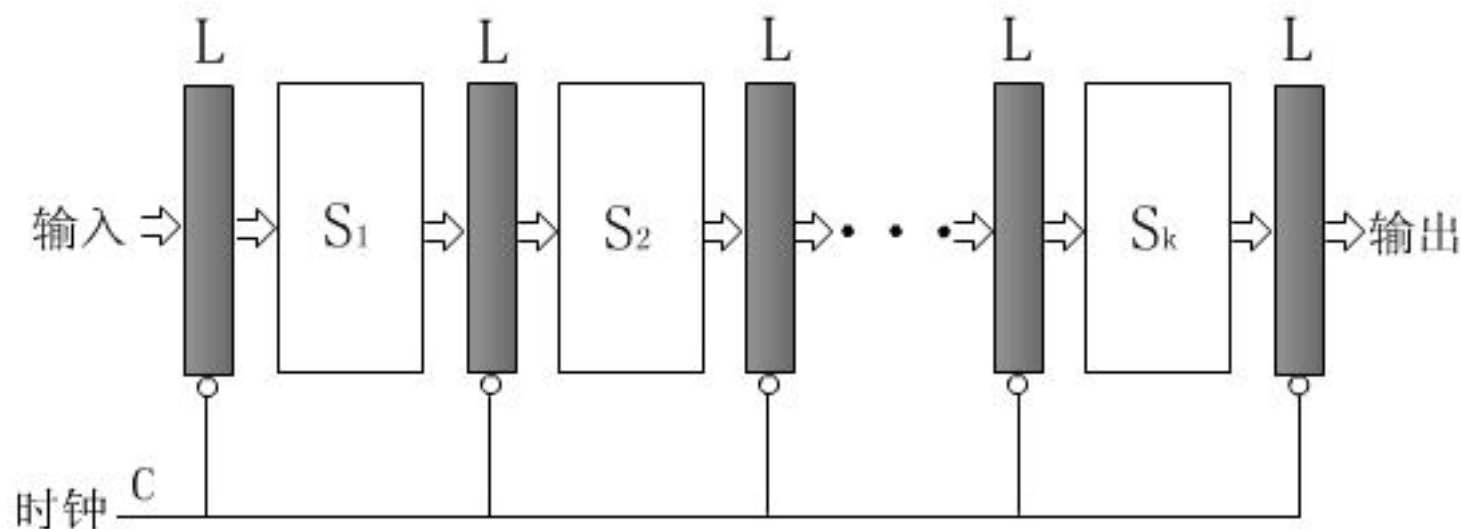
流水硬件基本结构

设阶段 S_i 所需的时间为 τ_i , 缓冲寄存器的延时为 τ_l ,
线性流水线的时钟周期定义为

$$\tau = \max \{ \tau_i \} + \tau_l = \tau_m + \tau_l$$

流水线处理的频率为

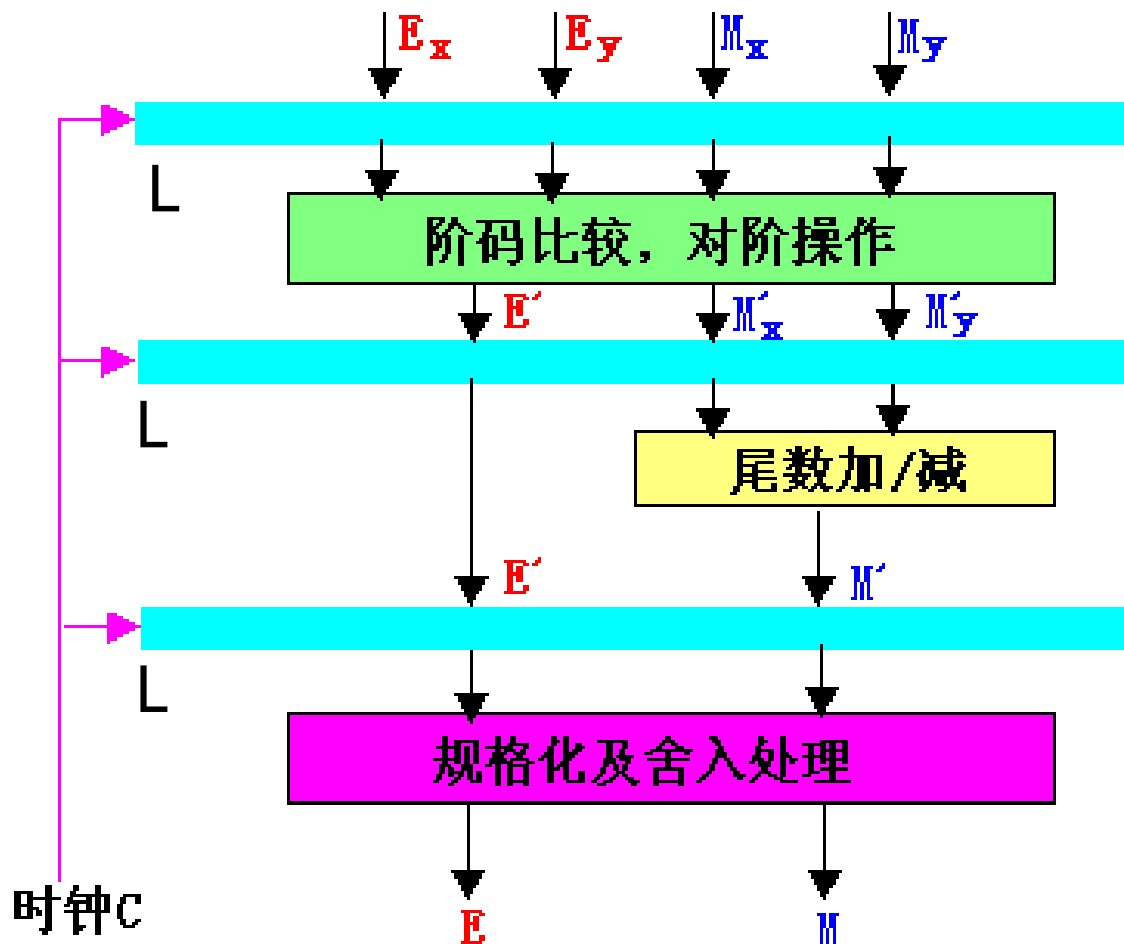
$$f = 1 / \tau$$





3段流水线浮点加法器 (0操作数检查除外)

■ 流水线浮点加法器框图



$$x = 1.1000 \times 2^2$$

$$y = 1.1100 \times 2^4$$

$$E_x = 2$$

$$E_y = 4$$

$$M_x = 1.1000$$

$$M_y = 1.1100$$

$$E' = 4$$

$$M'_x = 0.0110 \quad M'_y = 1.1100$$

$$E' = 4 \quad M' = 10.0010$$

$$E = 5 \quad M = 1.0001$$





例32 (1)

假设有一个4级流水浮点加法器每个过程段所需的时间为：零操作数检查 $\tau_1=70\text{ns}$ ，对阶 $\tau_2=60\text{ns}$ ，相加 $\tau_3=90\text{ns}$ ，规格化 $\tau_4=80\text{ns}$ ，缓冲寄存器L的延时时间为 $\tau_5=10\text{ns}$ ，求：

- (1) 4级流水加法器的加速比为多少？
- (2) 如果每个过程段的时间都相同。即都为75ns（包括缓冲寄存器时间）时，加速比是多少？





例（2）

解：加法器的流水线时钟周期至少为：

$$\tau = 90 + 10 = 100\text{ns}$$

不用流水线，所需时间为：

$$\tau_1 + \tau_2 + \tau_3 + \tau_4 = 300\text{ns}$$

$$\text{加速比为: } C_k = 300/100 = 3$$

若每过程段均为75ns（包括缓冲寄存器时间10ns），则不包括L的过程段时间为65ns

加速比为：

$$C_k = 65 * 4 / 75 = 52/15$$



浮点运算器实例

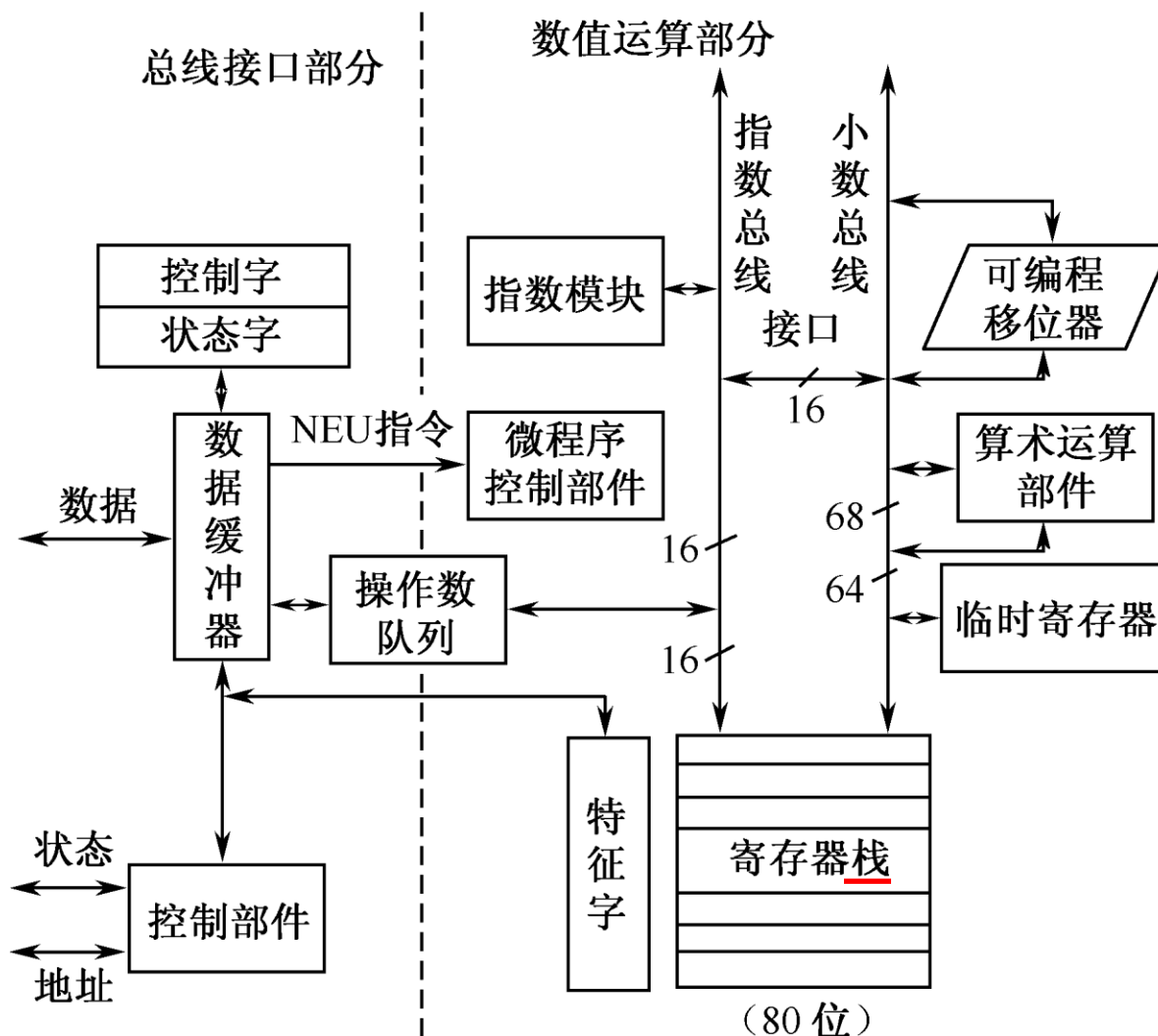
- CPU之外的浮点运算器（数学协处理器）如80x87
 - ◆ 完成浮点运算功能，不能单独使用
 - ◆ 可以和80386或80286异步并行工作
 - ◆ 使用特殊的80位的扩展精度格式。有8个**80位寄存器**组。IEEE754标准单精度和双精度数从存储器加载到浮点寄存器中时转化成这种格式
- CPU之内的浮点运算器（486DX以上）

较新的Intel处理器提供对754标准的直接硬件支持

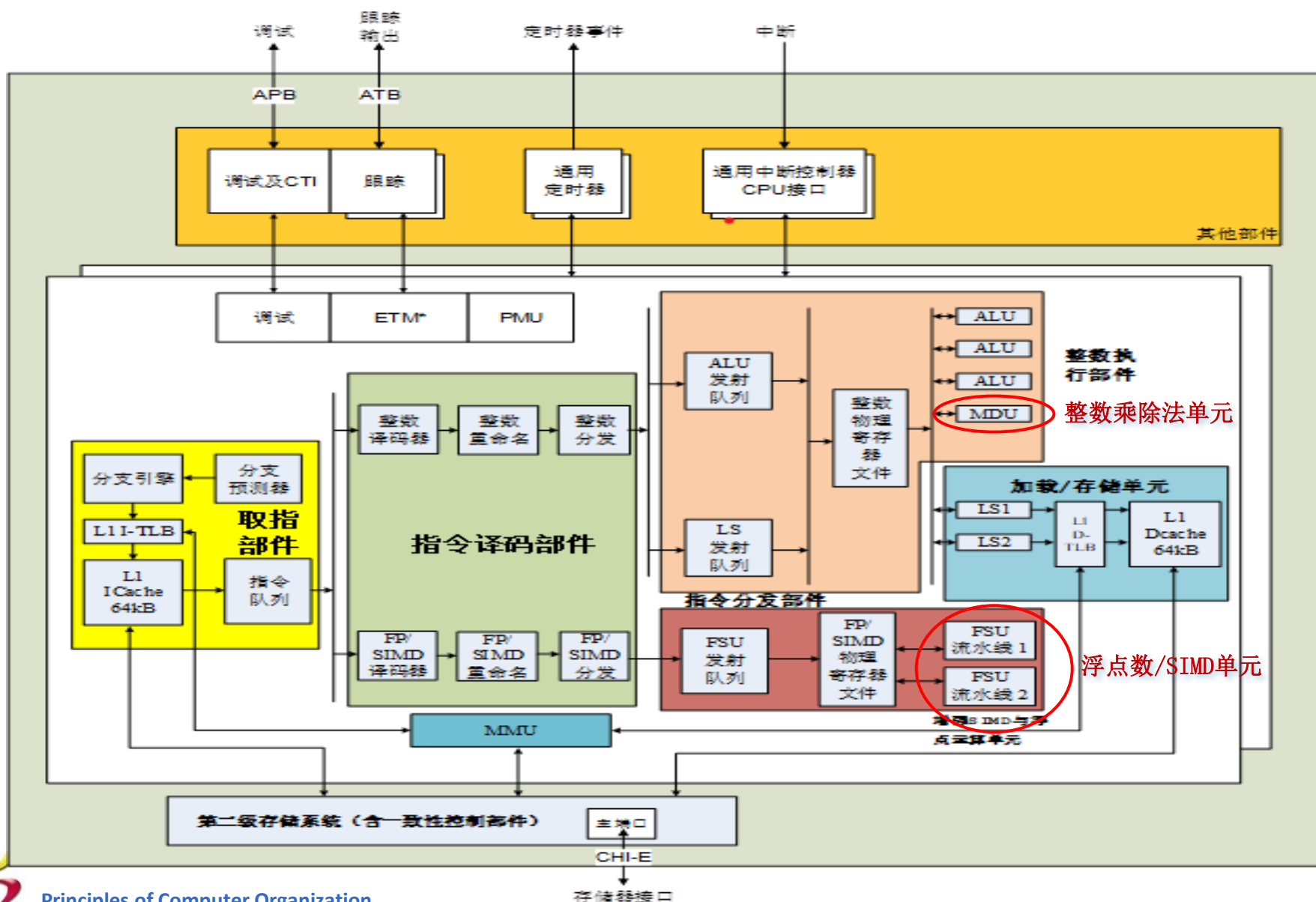




80x87浮点运算器逻辑框图



TaiShanV110处理器内核的功能结构





本章小结

- 运算器功能：处理数据
- 数据表示：原码、反码、补码、移码
- 功能实现：加、减、乘、除
- 运算器的构成
- 浮点数的表示及运算
- 浮点运算器

